

Trabalho Final - Computação Gráfica

Elisandro Raizer da Cruz Junior e Felipe Kenji Nishioka Jojima

Joinville, 07 de junho de 2025

Relatório

Este trabalho final apresenta o desenvolvimento de um mini game para destruir espaçonaves utilizando PyOpenGL e Pygame, demonstrando conhecimentos adquiridos nas disciplinas de Computação Gráfica e Programação de Jogos. O jogo envolve a renderização gráfica em 2D, controle de eventos de teclado, além da implementação de lógica para movimentação de inimigos e detecção de colisões.

Foi aplicada a biblioteca PyOpenGL para renderização eficiente das formas e efeitos gráficos (texturas, principalmente - usando GL_TEXTURE_2D e GL_QUADS), enquanto o Pygame foi utilizado para gerenciamento da janela e entrada do usuário. O objetivo foi criar um jogo simples, porém funcional, que demonstra a integração entre manipulação gráfica de baixo nível e a lógica interativa de jogos.

Para a execução deve-se executar a seguinte linha de comando:

python game.py

ou

python3 game.py

Tarefas

As tarefas foram sendo desenvolvidas conforme a agenda de cada membro da equipe, sendo repassado os arquivos de código a cada iteração e alteração feita.

20/06 das 11 horas até às 11:30 horas (30 minutos) – Configuração do Ambiente e Renderização Básica (Elisandro)

Esta etapa inicial é refletida nas primeiras linhas do código, onde o ambiente é preparado.

- **Inicialização:** As linhas `pygame.init()` e `glutInit()` preparam as bibliotecas.
- **Criação da Janela:** `pygame.display.set_mode(display, DOUBLEBUF | OPENGL)` cria a janela do Pygame com um contexto OpenGL, permitindo que os comandos `gl*` desenhem nela.
- **Configuração do OpenGL:** O código abaixo configura a câmera para uma visão 2D, onde o mundo do jogo vai de -1 a 1 em ambos os eixos (X e Y). Isso corresponde exatamente à função `init_gl()` mencionada.

```
pygame.init()
glutInit()

display = (800, 600)
pygame.display.set_mode(display, DOUBLEBUF | OPENGL)
pygame.display.set_caption("Nave com PyOpenGL")
```

```
# Setup OpenGL
glMatrixMode(GL_PROJECTION)
glLoadIdentity()
gluOrtho2D(-1, 1, -1, 1)
glMatrixMode(GL_MODELVIEW)
```

Figuras 1 e 2: Inicialização da janela e do OpenGL.

22/06 das 15 horas até às 18 horas (3 horas) – Desenvolvimento da Nave (Felipe)

A nave do jogador é gerenciada por variáveis e funções simples, em vez de uma classe `Player`.

- **Posição e Movimento:** A posição da nave é armazenada em `nave_pos`. O movimento é tratado dentro do loop principal, lendo as teclas `K_LEFT` e `K_RIGHT` e atualizando a posição, exatamente como descrito.
- **Desenho da Nave:** A função `draw_nave()` é responsável por desenhar a nave na tela na sua posição atual.

```
def draw_nave(x, y):
    # Ajuste de escala para caber na tela - você pode ajustar esse valor
    scale = 0.1
    w = largura_nave / 800 * 2 * scale # normaliza para espaço OpenGL [-1,1]
    h = altura_nave / 600 * 2 * scale
    draw_textured_quad(x, y, w, h, textura_nave)
```

Figura 3: `draw_nave()` atual. (`draw_texture_quad()` foi implementada e adicionada à função posteriormente)

```

keys = pygame.key.get_pressed()
if keys[K_LEFT] and nave_pos[0] - nave_vel - 0.05 > -1:
    nave_pos[0] -= nave_vel
if keys[K_RIGHT] and nave_pos[0] + nave_vel + 0.05 < 1:
    nave_pos[0] += nave_vel
if keys[K_SPACE] and (frame_count - last_shot_frame > shot_cooldown):
    shots.append([nave_pos[0], nave_pos[1] + 0.12])
    last_shot_frame = frame_count

```

Figura 4: binds relacionadas ao movimento e ações da nave - O K_SPACE foi adicionado posteriormente junto da mecânica de tiros.

25/06 das 14 horas até às 18 horas (4 horas) – Criação dos Inimigos (Elisandro)

Assim como a nave, os inimigos são gerenciados por uma lista e por funções.

- **Geração de Inimigos:** Uma nova nave inimiga é adicionada à lista `inimigos` em intervalos regulares, controlados por `frame_count` e `spawn_interval`. A posição inicial X é aleatória.
- **Movimentação e Desenho:** No loop principal, a posição Y de cada inimigo é atualizada (`inimigo[1] -= 0.01`), e a função `draw_inimigo()` é chamada para desenhar cada um na tela.

```

# Gerar inimigos
if frame_count % spawn_interval == 0:
    inimigos.append([random.uniform(-0.9, 0.9), 1.0])

# Atualizar inimigos
novos_inimigos = []
for inimigo in inimigos:
    inimigo[1] -= 0.01
    if inimigo[1] > -1:
        novos_inimigos.append(inimigo)
    else:
        vidas -= 1 # perdeu vida
inimigos = novos_inimigos

```

Figura 5: Funções para gerar novos inimigos e para atualizar eles.

26/06 das 14 horas até às 16 horas (2 horas) – Implementação dos Projéteis e Colisão (Felipe)

Esta funcionalidade é crucial para o gameplay e está bem implementada.

- **Disparo:** Ao pressionar `K_SPACE`, um novo tiro é adicionado à lista `shots`. Um `cooldown` (`shot_cooldown`) impede o disparo contínuo.
- **Deteção de Colisão:** A função `colisao(shot, inimigo)` verifica a proximidade entre um tiro e um inimigo. O loop principal usa essa função para verificar todos os tiros

contra todos os inimigos. Se uma colisão ocorrer, o inimigo e o tiro são removidos das suas respectivas listas.

```
def colisao(shot, inimigo):
    sx, sy = shot
    ix, iy = inimigo
    return abs(sx - ix) < 0.08 and abs(sy - iy) < 0.07
```

Figura 6: função de colisão do inimigo com a bala.

30/06 das 10 horas até às 12 horas e das 17 horas até às 20 horas (5 horas) – Aprimoramento da Renderização com Texturas (Elisandro)

Este é um dos aspectos mais visíveis do código. Em vez de formas coloridas, o jogo usa imagens (texturas).

- **Carregamento de Texturas:** A função `carregar_textura()` é a peça central. Ela usa o Pygame para carregar um arquivo de imagem (.png), convertendo os dados da imagem para um formato que o OpenGL entende e cria uma textura OpenGL.
- **Aplicação de Texturas:** Funções como `draw_nave()`, `draw_inimigo()`, `draw_shot()` e `desenhar_fundo()` usam o ID da textura carregada para desenhar um quadrado (`GL_QUADS`) com a imagem correspondente aplicada sobre ele.

```
def carregar_textura(path):
    if not os.path.isfile(path):
        raise FileNotFoundError(f"Arquivo de textura não encontrado: {path}")
    textura_surface = pygame.image.load(path)
    textura_data = pygame.image.tostring(textura_surface, "RGBA", True)
    largura, altura = textura_surface.get_rect().size
    textura_id = glGenTextures(1)
    glBindTexture(GL_TEXTURE_2D, textura_id)
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE)
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE)
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR)
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR)
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, largura, altura, 0, GL_RGBA, GL_UNSIGNED_BYTE, textura_data)
    glBindTexture(GL_TEXTURE_2D, 0)
    return textura_id, largura, altura
```

Figura 7: Função `carregar_textura()`

```
def draw_textured_quad(x, y, width, height, textura_id):
    glEnable(GL_TEXTURE_2D)
    glBindTexture(GL_TEXTURE_2D, textura_id)
    glColor3f(1,1,1)
    glBegin(GL_QUADS)
    glVertex2f(0, 0)
    glVertex2f(x - width/2, y)
    glVertex2f(1, 0)
    glVertex2f(x + width/2, y)
    glVertex2f(1, 1)
    glVertex2f(x + width/2, y + height)
    glVertex2f(0, 1)
    glVertex2f(x - width/2, y + height)
    glEnd()
    glBindTexture(GL_TEXTURE_2D, 0)
    glDisable(GL_TEXTURE_2D)
```

Figura 8: Função que `draw_textured_quad()`.

03/07 das 16 horas até às 19 horas (3 horas) – Sistema de Pontuação e Vidas (Felipe)

Os elementos de feedback para o jogador (HUD) e as condições de vitória/derrota estão presentes.

- **Vidas:** A variável `vidas` começa em 3. Uma vida é perdida quando um inimigo chega ao final da tela (`inimigo[1] <= -1`). O jogo termina quando `vidas` chega a 0.
- **Pontuação:** A variável `pontuacao` é incrementada em 10 cada vez que uma colisão entre um tiro e um inimigo é detectada.
- **Exibição na Tela:** As funções `draw_text()` e `glutStrokeCharacter` são usadas para desenhar o número de vidas e a pontuação no canto da tela a cada quadro.

05/07 das 15 horas até às 19 horas (4 horas) – Testes e Correções de Bugs (Elisandro)

O código final mostra sinais claros de refinamento.

- **Lógica de Remoção:** A maneira como os inimigos e tiros são removidos após a colisão (criando uma nova lista `inimigos_restantes`) é uma abordagem robusta que evita bugs comuns ao modificar uma lista enquanto se itera sobre ela.
- **Ajuste de Colisão:** Os valores "mágicos" na função `colisao()` (0.08 e 0.07) são o resultado direto de testes para encontrar um balanço que "pareça certo" no jogo.
- **Estabilidade:** O uso de `clock.tick(60)` no loop principal garante que o jogo rode a uma taxa de quadros estável (60 FPS), proporcionando uma experiência consistente.

06/07 das 18 horas até às 22:30 horas (3:30 horas) – Finalização e Documentação (Felipe e Elisandro)

O projeto está bem estruturado como um jogo completo.

- **Fluxo do Jogo:** O código não apenas joga, mas tem um ciclo completo: uma **tela de instruções** (`tela_instrucoes`), o **loop principal do jogo**, e uma **tela de game over** (`tela_game_over`) com opções para reiniciar ou sair. Isso torna a experiência do usuário coesa.
- **Modularidade:** A separação de responsabilidades em funções (`draw_nave`, `carregar_textura`, `colisao`, etc.) torna o código mais limpo e fácil de entender, o que reflete uma boa organização.

```

def tela_instrucoes():
    textura_instrucoes, largura_instr, altura_instr = carregar_textura("./png/instrucoes.png")

    esperando = True
    while esperando:
        glClear(GL_COLOR_BUFFER_BIT)

        # Desenha a textura de instruções ocupando toda a tela
        glLoadIdentity()
        glEnable(GL_TEXTURE_2D)
        glBindTexture(GL_TEXTURE_2D, textura_instrucoes)
        glColor3f(1, 1, 1)
        glBegin(GL_QUADS)
        glTexCoord2f(0, 0)
        glVertex2f(-1, -1)
        glTexCoord2f(1, 0)
        glVertex2f(1, -1)
        glTexCoord2f(1, 1)
        glVertex2f(1, 1)
        glTexCoord2f(0, 1)
        glVertex2f(-1, 1)
        glEnd()
        glBindTexture(GL_TEXTURE_2D, 0)
        glDisable(GL_TEXTURE_2D)

        pygame.display.flip()

        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                return False # Fecha o jogo
            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_RETURN: # Aperta Enter para começar
                    esperando = False
                    return True

        pygame.time.wait(10)

```

Figura 9: tela de instruções (início).

```

def tela_game_over():
    while True:
        glClear(GL_COLOR_BUFFER_BIT)
        glColor3f(1, 0, 0)
        desenhar_fundo(textura_gameover, largura_gameover, altura_gameover)

        pygame.display.flip()

        for event in pygame.event.get():
            if event.type == QUIT:
                return False # fechar o jogo
            if event.type == KEYDOWN:
                if event.key == K_r:
                    return True # reiniciar o jogo
                elif event.key == K_q:
                    return False # fechar o jogo

        pygame.time.wait(100)

```

Figura 10: tela de game over.